

WFLOW-99: Best Practice Summary

Series: WFLOW — Workflows and Alert Notifications | Notebook: 99 | Created: March 2026 | Last Updated: 10/06/2026

Overview

This notebook consolidates every actionable best practice from the WFLOW series (notebooks 01-09 and the WFLOW-94 LAB) into definitive tables organized by category. Each best practice includes the exact setting or value to use, a priority rating, and the source notebook.

Table of Contents

- 1. [Workflow Design & Architecture](#)
- 2. [Triggers](#)
- 3. [Connections & Credentials](#)
- 4. [Notification Channels](#)
- 5. [Message Formatting & Templates](#)
- 6. [Routing & Escalation](#)
- 7. [Incident Management Integration](#)
- 8. [Auto-Remediation](#)
- 9. [JavaScript & HTTP Actions](#)
- 10. [Security](#)
- 11. [Access Control](#)
- 12. [Observability & Monitoring](#)
- 13. [Operations & Change Management](#)

1. Workflow Design & Architecture

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use Workflows for all new automation	Problem-triggered workflows rather than Classic alerting profiles + problem notifications. No end-of-life date is published for profiles, but management-zone scoping does not carry over: rebuild it with affected-entity tags or a custom DQL matcher	Critical	WFLOW-01
2	Use a simple workflow for a single-destination notification	One task, a restricted action set (no Run JavaScript, no Run workflow) and no workflow-hours cost	Recommended	WFLOW-01, WFLOW-03

#	Best Practice	Recommended Setting/Value	Priority	Source
3	Use a standard workflow only when you need one	Several tasks, branches, loops or JavaScript make a standard workflow, billed in workflow hours for as long as it exists, not per run. Delete experiments you no longer need	Recommended	WFLOW-01, WFLOW-03
4	Deploy before you wait for a trigger	Triggers only start live workflows; a draft runs only from the Run button. Select Deploy	Critical	WFLOW-01, WFLOW-02
5	Run production workflows as a service user	Every task runs with the actor's permissions; set a service user as the actor of production workflows that several people work on	Critical	WFLOW-01, WFLOW-09
6	Keep workflows small and focused	One trigger → one outcome. In community practice, stay under ~20 tasks and split larger flows; no per-workflow task cap is published	Recommended	WFLOW-01
7	Set explicit task timeouts	Default 60 minutes, max 7 days: <code>timeout: <seconds></code> per task. Distinct from the 120 s runtime budget per action (DQL query, JavaScript/HTTP call), which cannot be raised	Critical	WFLOW-01, WFLOW-08
8	Design within the documented limits	1,000 event-triggered executions per hour per workflow (beyond that, HTTP 429 throttling); 1,000-character trigger filter; 10 MB input and result; 10,000 workflows per environment. Concurrent executions run <i>within system capacity</i> , with no fixed cap published	Recommended	WFLOW-01, WFLOW-09
9	Start with simple notifications	Basic notifications first, then layer in complexity	Recommended	WFLOW-09
10	Run independent notification tasks in parallel	Tasks with no predecessor start in parallel when the trigger fires (Slack + Teams + Email)	Recommended	WFLOW-03
11	Chain dependent tasks with predecessors	A task with predecessors waits for all of them; give it a state condition on each (<code>SUCCESS</code> , <code>OK</code> , <code>ERROR</code> , <code>NOK</code> , <code>ANY</code>)	Recommended	WFLOW-04, WFLOW-08

2. Triggers

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use the Problem trigger for incident alerts	Trigger type <i>Problem</i> (<code>davis-problem</code> in an exported workflow); its payload is the <code>dt.davis.problems</code> record	Critical	WFLOW-02
2	Use the Davis event trigger only for per-alert reactions	Put the threshold in an anomaly detector (ALERT-02); trigger on the problem, and use the Davis event trigger only when you need per-alert granularity	Critical	WFLOW-02
3	Grant the actor read access to what the trigger watches	<code>storage:events:read</code> for Problem and Davis event triggers; <code>storage:bizevents:read</code> , <code>storage:security.events:read</code> or <code>storage:system:read</code> for Event triggers on those tables; plus <code>storage:buckets:read</code>	Critical	WFLOW-02

#	Best Practice	Recommended Setting/Value	Priority	Source
4	Use the Schedule trigger for reports and health checks	Fixed time, time interval, or a five-field cron expression (six-field Quartz expressions are not supported)	Critical	WFLOW-02
5	Test with a manual run before deploying	Run in the editor works on a draft; <code>POST /platform/automation/v1/workflows/<id>/run</code> (platform token with <code>automation:workflows:run</code>) runs the live workflow	Critical	WFLOW-02
6	Scope Problem triggers with affected-entity tags	<i>Include entities with all defined tags</i> (<code>entityTagsMatch: all</code> , <code>entityTags: {env: [prod]}</code> in an export). The trigger has no management-zone filter	Critical	WFLOW-02
7	Select only the problem categories you need	Availability, Error, Slowdown, Resource, Custom, Monitoring unavailable, Info (there is no Performance category); add the Severity filter to drop lower levels (WFLOW-04 §3). Every selection uses part of the 1,000-character filter budget	Recommended	WFLOW-02, WFLOW-04
8	Enable Wait for root cause analysis	Advanced options. Dynatrace marks it <i>Recommended</i> to avoid triggering on incomplete problem data	Recommended	WFLOW-02
9	Use Minimum duration to suppress short-lived problems	5, 10, 15, 30, 60, 120, 240, 1440 or 10080 minutes; the replacement for an alerting profile's delay. A problem that closes sooner never starts the workflow	Recommended	WFLOW-02, WFLOW-04
10	Handle all problem lifecycle events	Problem state <i>active or closed</i> ; branch on <code>event.status</code> (<code>ACTIVE</code> / <code>CLOSED</code>) in a single workflow	Recommended	WFLOW-05
11	Validate the filter with Query past events	Zero matches across every window in an environment you know is busy means the filter is over-constrained; nothing else reports it	Recommended	WFLOW-02
12	Stagger scheduled workflows	Start minutes such as <code>:00</code> , <code>:03</code> , <code>:07</code> instead of pinning every workflow to <code>:00</code>	Recommended	WFLOW-02
13	Always set a time zone on schedule triggers	<code>timezone: America/New_York</code> ; a cron schedule otherwise takes the browser's locale	Recommended	WFLOW-02
14	Use the Event trigger for business process automation	Event type <code>bizevents</code> with a DQL-matcher filter query (<code>event.type == "com.company.order-completed"</code>); send business events as flat JSON	Optional	WFLOW-02

3. Connections & Credentials

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use descriptive connection names	Pattern: <service>-<environment>-<purpose> (e.g., slack-prod-oncall)	Critical	WFLOW-03
2	Separate connections per environment	Distinct connections for prod vs staging vs dev	Critical	WFLOW-03
3	Apply least-privilege scopes	Slack bot: chat:write, channels:read and groups:read (Dynatrace's minimal manifest); add chat:write.public or channels:join only to post to channels the bot has not joined	Critical	WFLOW-03, WFLOW-09
4	Allow every destination host	A host pattern under Settings > General > External requests for each connector and HTTP target, and the connector permissions under Workflows > Settings > Authorization settings	Critical	WFLOW-03
5	Rotate credentials with a dual-credential overlap	Cadence set by your security policy (community baseline: quarterly for Slack bot tokens and ServiceNow OAuth secrets, annually for PagerDuty integration keys). Mint the new credential, create a -v2 connection, migrate workflows, then revoke the old credential	Recommended	WFLOW-09
6	Prefer OAuth Client Credentials for ServiceNow	The connection accepts only Basic Authentication or OAuth Client Credentials. Use a dedicated integration user with table-level rights, never admin	Recommended	WFLOW-05, WFLOW-09
7	Pick the PagerDuty connection by action	An Events connection (routing key) for Send event ; a REST API key connection for Create an incident and the list actions. Separate Events connections per routing target (pagerduty-prod-critical, pagerduty-prod-standard, pagerduty-platform)	Recommended	WFLOW-05
8	Use a bot token for the Slack connection	The Slack connector takes a Slack app bot token and has no webhook option. An incoming webhook is reachable only from an HTTP Request task	Recommended	WFLOW-03

4. Notification Channels

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Multi-channel for Critical production problems	event.severity 1: PagerDuty + Slack #alerts-urgent + Email simultaneously	Critical	WFLOW-03, WFLOW-04
2	Slack + email for Major production problems	event.severity 2: Slack #alerts-production + Email	Recommended	WFLOW-04
3	Slack only for Minor problems	event.severity 3, or unset: Slack #alerts-production; staging gets a daily digest. Problems never carry 4 or 5	Recommended	WFLOW-04
4	Use a Teams Workflows (Power Automate) webhook	Office 365 connectors stopped working in May 2026. The Teams connection takes the URL from <i>Send webhook alerts to a channel</i> ; allow its host (e.g., *.api.powerplatform.com) under External requests	Critical	WFLOW-03

#	Best Practice	Recommended Setting/Value	Priority	Source
5	Use the built-in Send email action for simple notifications	Sends from <code>no-reply@apps.dynatrace.com</code> , up to 10 addresses per To/Cc/Bcc field, markdown rather than HTML, and needs <code>email:emails:send</code> . It has no SMTP option: for a corporate sender or high volume, call the provider's API (SendGrid, SES, a relay) from an HTTP Request task	Optional	WFLOW-03
6	Include the problem category in the email subject	<code>{{ event()['event.category'] }}</code> <code>{{ event()['event.name'] }}</code>	Recommended	WFLOW-03

5. Message Formatting & Templates

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Structure messages: Severity + Title + Key metrics + Link	Line 1: severity indicator + title; Line 2: impact; Line 3: affected entities; Line 4: action link	Critical	WFLOW-06
2	Use severity-coded visual indicators	Slack, keyed on <code>event.severity</code> : <code>:red_circle:</code> 1 Critical, <code>:large_orange_circle:</code> 2 Major, <code>:large_yellow_circle:</code> 3 Minor, <code>:large_blue_circle:</code> 4 Warning (Davis events only; problems carry 1–3)	Critical	WFLOW-06
3	Use Slack Block Kit for rich messages	Block Kit JSON in <code>message</code> with <code>messageFormat: slack_format</code> ; the action has no separate <code>blocks</code> input. A double quote in a resolved value breaks the JSON, so send a test message	Recommended	WFLOW-03, WFLOW-06
4	Use Teams Adaptive Cards v1.4	Card JSON (<code>type: AdaptiveCard</code> , <code>version: "1.4"</code> , <code>FactSet</code> , <code>TextBlock</code> , <code>Action.OpenUrl</code>) in <code>message</code> with the default <code>msteams_format</code> , or a <code>selectTemplate</code> card. Use Dynatrace expressions; Adaptive Card templating is not supported	Recommended	WFLOW-03, WFLOW-06
5	Always include a link to the Dynatrace problem	<code><{{ problem_link() }}\ View in Dynatrace></code> (Slack) or <code>Action.OpenUrl</code> (Teams). <code>problem_link()</code> evaluates only under a Problem trigger	Critical	WFLOW-03
6	Use dictionary mapping for severity-to-emoji	<code>{{ {1: ":red_circle:", ...}.get(event().get("event.severity") \ int(5), ":white_circle:") }}</code>	Recommended	WFLOW-06
7	Limit affected entities shown to 3-5	Read Smartscape names first and fall back to the deprecated classic IDs, which are better populated today: <code>(event().get("smartscape.affected_entities") or []) \ map(attribute="name") \ join(", ")</code> or <code>event()["affected_entity_ids"] \ join(", ")</code> . Slice to <code>[:3]</code> with an overflow indicator	Recommended	WFLOW-05, WFLOW-06

#	Best Practice	Recommended Setting/Value	Priority	Source
8	Read optional fields with <code>.get()</code>	<code>(event().get("root_cause.smartscape_entity") or {}).get("name", "not yet determined")</code> . Most problems name no root cause (79% had neither <code>root_cause.smartscape_entity</code> nor the deprecated <code>root_cause_entity_id</code> over 7 days on a validation tenant, 10/06/2026), and a bracket read of a missing field fails the task with <i>Undefined variables</i>	Critical	WFLOW-02, WFLOW-03, WFLOW-07
9	Truncate long strings	<code>{{ event()["event.name"] \ truncate(50) }}</code>	Recommended	WFLOW-06
10	Enrich notifications with DQL data	A JavaScript task before the notification task runs <code>queryExecutionClient.queryExecute()</code> and polls with <code>queryPoll()</code> until the query finishes; skip it when the problem names no root cause	Optional	WFLOW-06, WFLOW-08
11	Test templates before production	An On-Demand workflow with a JavaScript mock event using the <code>dt.davis.problems</code> field names. <code>problem_link()</code> renders only under a Problem trigger, so confirm the link on a real problem. A template that fails to evaluate shows up as <code>Error evaluating</code> on the <code>WORKFLOW_EXECUTION</code> record	Critical	WFLOW-06
12	Preview Slack templates in Block Kit Builder	https://app.slack.com/block-kit-builder before deploying	Recommended	WFLOW-06
13	Create a resolved-problem template	<code>:white_check_mark: Problem Resolved</code> with the duration from <code>resolved_problem_duration</code> and the resolution time from <code>event.end</code>	Recommended	WFLOW-06

6. Routing & Escalation

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Route by severity	<code>event.severity</code> 1 Critical: PagerDuty + Slack + Email; 2 Major: Slack + Email; 3 Minor, or unset: Slack. Problems never carry 4 (Warning) or 5 (Informational). Check which sources set severity: SaaS 1.348 stops defaulting it to 3 (pre-release, staged rollout)	Critical	WFLOW-04
2	Route by team ownership tags	Preferred: one workflow per team, whose Problem trigger filters on the built-in ownership tag (<code>owner / dt.owner</code>) with the <i>Affected entities</i> tag filter, so the match is made before any task runs. Fallback in a task condition: on problem records <code>tags</code> is a JSON-encoded string, so <code>"owner:checkout" in event().get("tags", "")</code> is a substring test (<code>owner:cart</code> also matches <code>owner:cart-v2</code>). Use tag values that are not prefixes of one another	Critical	WFLOW-04
3	Do not route by management zone	<code>"Production" in event()["management_zones"]</code> is legacy, and the field is not present on problem records (0 of 15,878 over 30 days on a live tenant, 10/05/2026, with management zones still in use), so the condition has nothing to match. Route on ownership tags or Smartscape ownership instead	Critical	WFLOW-04

#	Best Practice	Recommended Setting/Value	Priority	Source
4	Implement time-based routing	Business hours: Slack channel; off-hours Critical: PagerDuty; off-hours non-critical: queue for morning. Pass the team's time zone, <code>now("America/New_York")</code> ; <code>now()</code> is UTC otherwise. Business calendars (<code>calendars()</code>) cover holidays	Critical	WFLOW-04
5	Escalate problems that are still active	Task option Wait before (<code>waitBefore: 900</code>), then re-query <code>dt.davis.problems</code> and escalate to PagerDuty if <code>event.status</code> is still <code>ACTIVE</code> ; or a second workflow with Minimum duration 15. Neither sees acknowledgment: leave acknowledgment-aware escalation to the paging tool	Recommended	WFLOW-04
6	Multi-tier escalation	0 min: Slack; 15 min: Email team lead; 30 min: PagerDuty on-call; 60 min: PagerDuty manager	Recommended	WFLOW-04
7	Combine checks in one custom condition	Each task has one <code>conditions</code> block: <code>states</code> of its predecessors, one <code>custom</code> Jinja expression with <code>and</code> / <code>or</code> / <code>not</code> inside it, and <code>else: SKIP</code> on branches that may not run (the default is stop). E.g. <code>custom: '{{ (event().get("event.severity") \ int(5)) <= 1 and "env:prod" in event().get("tags", "") }}'</code>	Recommended	WFLOW-04
8	Use JavaScript for dynamic channel selection	Read the <code>owner: tag</code> from <code>entity_tags</code> (deprecated; may arrive as an array or a JSON string) to build <code>#{team}-alerts</code> , or use the Ownership app's Get owners action for each team's Slack channels	Optional	WFLOW-04, WFLOW-08
9	Separate environment routing	Production: immediate notification; Staging: Slack only; Dev: daily digest	Recommended	WFLOW-04

7. Incident Management Integration

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Deduplicate PagerDuty incidents	<code>dedupKey: "dynatrace-{{ event()['display_id'] }}"</code> on every Send event for the problem, through the same Events connection	Critical	WFLOW-05
2	Deduplicate ServiceNow incidents	Set <code>correlationId: "DT-{{ event()['display_id'] }}"</code> and run Search incidents on it before Create Incident , creating only when nothing is found. Create Incident is a plain POST and does not deduplicate on the correlation ID	Critical	WFLOW-05
3	Map Dynatrace severity to PagerDuty severity	<code>event.severity 1: critical; 2: error; 3: warning; 4: info</code>	Critical	WFLOW-05
4	Map Dynatrace severity to ServiceNow impact/urgency	<code>event.severity 1-2: impact=1, urgency=1; 3: impact=2, urgency=2; 4: impact=3, urgency=3; 5: usually no incident</code>	Critical	WFLOW-05
5	Auto-resolve incidents when the problem closes	Problem state <i>active</i> or <i>closed</i> . On <code>event.status == "CLOSED"</code> : Send event with <code>eventAction: resolve</code> and the same <code>dedupKey</code> (PagerDuty), or Resolve incident with the incident number from Search incidents, resolution notes and a resolution code from your instance's list (ServiceNow)	Critical	WFLOW-05

#	Best Practice	Recommended Setting/Value	Priority	Source
6	Include Dynatrace problem URL in incident details	<code>{{ problem_link() }}</code> in <code>customDetails</code> (PagerDuty) or the <code>description body</code> (ServiceNow)	Critical	WFLOW-05
7	Write the incident number back onto the problem	Read <code>result("create_snow_incident").number</code> downstream and post it with <code>problemsClient.createComment()</code> on <code>event()["event.id"]</code> . Comment and Resolve take the incident number, not the <code>sys_id</code>	Recommended	WFLOW-05
8	Use PagerDuty Events API v2	Integration type: Events API v2 (not v1)	Critical	WFLOW-05
9	Fill ServiceNow's required fields	Category, Subcategory, Impact, Urgency and Assignment Group are required on Create Incident. Pick the assignment group in the editor (it is stored as the group's <code>sys_id</code>) and set Caller	Critical	WFLOW-05
10	Implement bi-directional sync	A scheduled workflow polls ServiceNow for open DT-correlated incidents, with a Credential Vault credential, and writes their status back as problem comments; or the ServiceNow-side Dynatrace app / ITOM (ALERT-04)	Optional	WFLOW-05
11	Harden the integration	Retry on error for transient failures (fixed count and delay, no backoff, retries every failure); route the task's error state to a dead-letter task; monitor connector <code>ACTION_EXECUTION</code> errors	Recommended	WFLOW-05

8. Auto-Remediation

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Rate-limit remediation attempts	Max 3 remediation runs per hour, counted with <code>countDistinctExact(dt.automation_engine.workflow_execution.id)</code> excluding the current run. The check task throws at the limit, so a remediation task gated on its <code>SUCCESS</code> does not run. The count spans all entities; a per-entity limit needs the entity recorded, for example in a business event	Critical	WFLOW-07
2	Enforce time-window guardrails	A <code>custom</code> condition on the remediation task, weekdays 06:00–22:00 in a named time zone (<code>now("Europe/Vienna")</code>), with <code>else: SKIP</code>	Critical	WFLOW-07
3	Implement cooldown periods	30 minutes minimum between remediation actions on the same entity	Critical	WFLOW-07
4	Capture pre-remediation state for rollback	Record current state before executing any change	Critical	WFLOW-07
5	Gate every action on a root-cause entity	Read <code>root_cause.smartscape_entity</code> first, fall back to the deprecated <code>root_cause_entity_id</code> , and end with a "skipped" result when neither is present. Most problems name none (79% over 7 days on a validation tenant)	Critical	WFLOW-07, WFLOW-08

#	Best Practice	Recommended Setting/Value	Priority	Source
6	Require human approval for production remediation	Slack Request approval action (copy its ID from the action picker) with a 30-minute task timeout; test what a Decline and a timeout leave before gating on it. Auto-approve only when a primary tag positively marks the problem non-production, so a missing tag fails closed to approval	Critical	WFLOW-03, WFLOW-07
7	Auto-remediate only well-defined scenarios	Pod crash loops: yes (restart the workload); Disk space 90%: yes (cleanup); Database deadlocks: no (investigate); Security incidents: no (human judgment)	Critical	WFLOW-07
8	Remediate through connectors	Kubernetes Connector Rollout restart resource on the one workload the problem names (it has no rollback action; leave rollback to your CD tooling), and AWS Connector Reboot instances after resolving the instance ID from Smartscape. A Run JavaScript task cannot import npm client libraries by name	Recommended	WFLOW-07
9	Follow remediation maturity model	Level 0: manual; Level 1: notification + runbook link; Level 2: semi-automated (approval); Level 3: fully automated with guardrails; Level 4: self-healing with learning	Recommended	WFLOW-07
10	Map problem types to runbooks	JavaScript <code>runbookMap</code> object mapping problem title patterns to runbook IDs	Recommended	WFLOW-07
11	Include runbook links in notifications	Jinja conditional: <code>{% if "CPU" in event()["event.name"] %}</code> links to CPU runbook	Recommended	WFLOW-07
12	Validate after remediation	Query metrics/logs post-action to confirm resolution	Recommended	WFLOW-07
13	Start non-prod, promote to prod	Test all remediation workflows in staging before enabling in production	Critical	WFLOW-07

9. JavaScript & HTTP Actions

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Always use <code>export default async function</code>	The runtime injects only <code>executionId</code> and <code>actionExecutionId</code> ; read the event with <code>execution()</code> and predecessor results with <code>await result()</code> from <code>@dynatrace-sdk/automation-utils</code>	Critical	WFLOW-08
2	Handle errors deliberately	<code>try / catch</code> for recoverable errors, returning a degraded result. Throw when the failure is terminal, so the task ends in <code>ERROR</code> and an on-failure task can route around it; a guardrail that returns <code>{proceed: false}</code> leaves its task in <code>SUCCESS</code>	Critical	WFLOW-07, WFLOW-08

#	Best Practice	Recommended Setting/Value	Priority	Source
3	Retry only what can succeed	Max 3 attempts, delay = <code>1000 * attempt</code> ms, on network errors and 5xx only (a 4xx is thrown at once). Retry a <code>POST</code> only if the target treats a repeat as a no-op	Recommended	WFLOW-08
4	Set request timeouts on external HTTP	<code>AbortController</code> with a 10-second timeout on outbound <code>fetch()</code> calls. Distinct from the task <code>timeout</code> (whole-task budget) and the 120 s runtime budget (per action). See WFLOW-08 §9–§10	Critical	WFLOW-08
5	Poll DQL queries to completion	<code>queryExecute()</code> returns the result only if the query finishes within <code>requestTimeoutMilliseconds</code> ; otherwise it returns a <code>requestToken</code> . Poll with <code>queryPoll()</code> until the state is final, and fail on anything but <code>SUCCEEDED</code> — never read <code>result.result.records</code> unchecked. The 120 s runtime budget still applies. See WFLOW-08 §2 and §10	Critical	WFLOW-08
6	Limit DQL query scope	<code>from: now() - 1h, select only needed fields, limit 100</code>	Critical	WFLOW-08
7	Use <code>Promise.all()</code> for parallel entity lookups	Execute all independent API calls concurrently	Recommended	WFLOW-08
8	Always use HTTPS for external calls	Never use <code>http://</code> in HTTP request URLs	Critical	WFLOW-09
9	Allow-list every outbound host	Add each host a Run JavaScript or HTTP Request task calls under Settings > General > External requests ; a call to an unlisted host fails with <code>Blocked request to '...' (host not in allowlist)</code> . Hosts on a private network, and targets that admit only allow-listed source IPs, are reached through EdgeConnect (WFLOW-94)	Critical	WFLOW-08, WFLOW-94
10	Read secrets from the Credential Vault	<code>credentialVaultClient.getCredentialsDetails({ id })</code> in JavaScript (credential scoped to AppEngine, <i>Allow access without app context</i> on, actor granted access); the Authentication field in HTTP tasks. Never hardcode, log or return a secret	Critical	WFLOW-08, WFLOW-09
11	Give each step its own failure task	A task waits for all of its predecessors, so one failure task chained after several steps never fires. Give each step a failure task with a single predecessor set to <code>NOK</code> and <code>else: SKIP</code>	Recommended	WFLOW-08
12	Add problem comments after remediation	<code>problemsClient.createComment()</code> with the problem ID <code>event.id</code> , not <code>display_id</code>	Recommended	WFLOW-08
13	Validate secret existence before use	<code>if (!apiToken) throw new Error('Missing required secret: ...')</code>	Recommended	WFLOW-09
14	Grant the workflow the scopes its tasks use	Workflows > Settings > Authorization settings (for example <code>environment-api:credentials:read</code> , <code>environment-api:problems:write</code> , the <code>storage:*:read</code> of each queried table); a missing scope fails the task with a 403	Recommended	WFLOW-07, WFLOW-08

10. Security

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Never hardcode secrets in workflows	Connections for connector actions; Credential Vault for HTTP (Authentication field) and JavaScript (<code>credentialVaultClient</code>)	Critical	WFLOW-09
2	Sanitize all dynamic inputs	Remove <code>"'\\" characters, limit string length to 200 chars before using in queries</code>	Critical	WFLOW-09
3	Never log secrets	Do not <code>console.log()</code> a credential or return it in task output — results are visible to anyone who can read the workflow	Critical	WFLOW-09
4	Name credentials consistently	Pattern: <code><service>-<env>-<purpose></code> for Credential Vault entries and connections	Recommended	WFLOW-09
5	Rotate secrets on your policy's cadence	Dual-credential overlap: mint the new credential, create a <code>-v2</code> connection, migrate workflows one at a time, wait a full execution cycle, then revoke the old one at its source. On suspected exposure, revoke first	Recommended	WFLOW-09
6	Fail secure on errors	Default to safe state (no action) when errors occur, never expose error details externally	Critical	WFLOW-09

11. Access Control

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Grant minimum permissions per role	Viewers: <code>automation:workflows:read</code> ; builders: <code>read + write + run</code> ; workflow admins add <code>automation:workflows:admin</code> (Workflow admin mode). Using Workflows needs <code>app-engine:apps:run</code> , and writing and running them <code>app-engine:functions:run</code> . Which workflows a user sees follows ownership and visibility, not a policy condition	Critical	WFLOW-01, WFLOW-09
2	Limit who can create billed workflows	<code>ALLOW automation:workflows:write WHERE automation:workflow-type = "SIMPLE"</code> ; for app teams. <code>automation:workflow-type</code> is the only condition on this permission; IAM has no workflow-owner condition	Recommended	WFLOW-09
3	Separate connection write access	Connections are settings objects: grant <code>settings:objects:write</code> on the connector's schema (e.g., <code>WHERE settings:schemaId = "app:dynatrace.pagerduty:connection"</code>) only to workflow admins and SRE. There is no <code>automation:connections:*</code> permission	Critical	WFLOW-09
4	Record ownership in the workflow's own fields	<code>owner</code> with <code>ownerType: GROUP</code> for team ownership, <code>isPrivate</code> , and a service-user actor; team and classification in the <code>description</code> or the naming convention. There is no free-form <code>metadata</code> ownership block	Recommended	WFLOW-09
5	Control who can assign service-user actors	<code>ALLOW iam:service-users:use WHERE iam:service-user-email IN ("<code><SERVICE_USER_EMAIL></code>")</code> ; . A person who edits a workflow becomes its actor unless the actor is a service user or the edit is made in Workflow admin mode	Critical	WFLOW-09

12. Observability & Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Alert on workflow success rate	Threshold: < 95% success rate over 7 days	Critical	WFLOW-09
2	Alert on average execution duration	Threshold: > 5 minutes average	Recommended	WFLOW-09
3	Alert on failure spike	Threshold: > 5 failures per hour	Critical	WFLOW-09
4	Build a workflow health dashboard	Queries on <code>dt.system.events (event.kind == "WORKFLOW_EVENT")</code> : overall health, per-workflow success rates, execution trend, recent failures, task-level performance. Count final records only (<code>dt.automation_engine.state.is_final == true</code>)	Critical	WFLOW-09
5	Create a workflow-monitoring workflow	Scheduled; query for workflows with 3+ failures in the last hour; alert to <code>#workflow-alerts</code> . Its actor needs <code>storage:system:read</code> and <code>storage:buckets:read</code>	Recommended	WFLOW-09
6	Monitor notification task success rates	Query final <code>ACTION_EXECUTION</code> records in <code>dt.system.events</code> by <code>dt.automation_engine.action.app (dynatrace.email , dynatrace.slack , ...)</code>	Recommended	WFLOW-03
7	Track remediation success rates	Query <code>WORKFLOW_EXECUTION</code> records filtered on <code>dt.automation_engine.workflow.title</code> containing <code>remediation</code> (<code>caseSensitive: false</code>), or on the remediation workflows' IDs	Recommended	WFLOW-07
8	Monitor skipped tasks	Query <code>TASK_EXECUTION</code> records with <code>dt.automation_engine.state SKIPPED / DISCARDED</code> to validate routing conditions are working as designed	Optional	WFLOW-04
9	Exclude tasks that never ran from duration statistics	<code>filter in(dt.automation_engine.state, {"SUCCESS", "ERROR"}) ; DISCARDED</code> and <code>SKIPPED</code> tasks are final but always carry duration 0	Recommended	WFLOW-09
10	Watch for throttling and blocked hosts	The Workflows overview's throttled filter (1,000 event-triggered executions per hour per workflow); Blocked request ... (host not in allowlist) in <code>dt.automation_engine.state_info</code> means a missing External requests entry	Recommended	WFLOW-09

13. Operations & Change Management

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Name workflows consistently	Pattern: <code><team>-<function>-<environment></code> (e.g., <code>sre-problem-notifications-prod</code>)	Critical	WFLOW-09

#	Best Practice	Recommended Setting/Value	Priority	Source
2	Export workflows for version control	<code>GET /platform/automation/v1/workflows/<id>/export</code> with a platform token (<code>automation:workflows:read</code> , sent as <code>Bearer</code>), saved to Git	Recommended	WFLOW-09
3	Test changes on cloned workflow first	Clone, modify, test with On-Demand trigger, review results, promote	Critical	WFLOW-09
4	Rollback procedure	Disable the failing workflow, restore the previous version from its history (<code>POST /platform/automation/v1/workflows/<id>/history/<version>/restore</code> , which deploys it), enable it, verify. Import your own export only if the history no longer has the version	Critical	WFLOW-09
5	Daily: check execution dashboard	Review success rates and investigate failures	Recommended	WFLOW-09
6	Weekly: verify external connections	Test all Slack, Teams, PagerDuty, ServiceNow connections	Recommended	WFLOW-09
7	Rotate secrets per your policy	Dual-credential overlap, then revoke the old credential (§10 row 5)	Recommended	WFLOW-09
8	Query execution history with DQL	<code>fetch dt.system.events, from:-24h \ filter event.kind == "WORKFLOW_EVENT" and event.type == "WORKFLOW_EXECUTION" \ filter dt.automation_engine.state.is_final == true — without is_final , a run is counted once as <code>RUNNING</code> and again in its final state</code>	Recommended	WFLOW-01

Summary

This notebook contains **128 best practices** across 13 categories extracted from the complete WFLOW series.

Priority distribution:

Priority	Count	Meaning
Critical	62	Must implement for production readiness
Recommended	60	Strongly advised for operational maturity
Optional	6	Beneficial for advanced use cases

Implementation order:

1. Connections, External requests host patterns and a service-user actor (WFLOW-01, WFLOW-03, WFLOW-09)
2. A simple problem-notification workflow, then severity and ownership routing (WFLOW-01, WFLOW-03, WFLOW-04)
3. Incident management integration with search-before-create deduplication (WFLOW-05)
4. Custom templates with severity indicators and links (WFLOW-06)

5. Workflow health monitoring dashboard (WFLOW-09)

6. Auto-remediation with guardrails (WFLOW-07)

References

- [Workflows umbrella \(DT docs\)](#)
- [Workflow triggers \(DT docs\)](#)
- [Workflow actions \(DT docs\)](#)
- [Notification actions umbrella \(DT docs\)](#)
- [HTTP request action \(DT docs\)](#)
- [Run JavaScript action \(DT docs\)](#)
- [Workflow reference / Jinja expressions \(DT docs\)](#)
- [Davis Problems app \(DT docs\)](#)
- [Alerting and notifications umbrella \(DT docs\)](#)
- [Upgrade guide — Alert notification \(DT docs\)](#)
- [Event triggers for workflows \(DT docs\)](#)
- [Build workflows \(DT docs\)](#)
- [Manage workflow permissions \(DT docs\)](#)
- [Automation Workflow consumption \(DT docs\)](#)
- [IAM policy statements \(DT docs\)](#)
- [Set up Slack Connector \(DT docs\)](#)
- [Email \(DT docs\)](#)
- [ServiceNow Connector \(DT docs\)](#)
- [EdgeConnect \(DT docs\)](#)
- [Dynatrace Developer Portal \(Dynatrace\)](#)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.